

Laboratório 3: Dados relacionais e operações do tipo join

Letícia da Costa Venancio - 250125

Carregando pacotes

```
library(readr)
library(dplyr)
```

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv("airports.csv", col_types = cols_only(
  IATA_CODE = col_character(),
  CITY      = col_character(),
  STATE     = col_character(),
  LATITUDE  = col_double(),
  LONGITUDE = col_double()
))
```

```
airlines <- read_csv("airlines.csv", col_types = cols_only(
  IATA_CODE = col_character(),
  AIRLINE   = col_character()
))
```

Para o arquivo de vôos:

a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;

- b. Leia apenas 1 milhão de linhas por vez;
- c. Remova vôos em que o aeroporto de destino comece com a letra 1;
- d. Remova registros em que existam pelo menos uma coluna faltante;
- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
cols_flights <- cols_only(  
  DESTINATION_AIRPORT = 'c',  
  ARRIVAL_DELAY       = 'i'  
)  
  
getStats <- function(input, pos){  
  input %>%  
    filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY)) %>% # (d)  
    filter(!startsWith(DESTINATION_AIRPORT, "1")) %>% # (c)  
    group_by(DESTINATION_AIRPORT) %>%  
    summarise(  
      soma = sum(ARRIVAL_DELAY), # (e)  
      n     = n(),  
      .groups = 'drop'  
    )  
}
```

```
# (b)  
stats_flights <- read_csv_chunked('flights.csv.zip',  
  callback    = DataFrameCallback$new(getStats),  
  chunk_size  = 1e6,  
  col_types   = cols_flights)
```

```
# (f)  
flights <- stats_flights %>%  
  group_by(DESTINATION_AIRPORT) %>%  
  summarise(soma = sum(soma),
```

```

      n      = sum(n),
      .groups = 'drop') %>%
mutate(atraso_medio = soma / n) %>%
select(DESTINATION_AIRPORT, atraso_medio)

```

```
flights
```

```

# A tibble: 322 x 2
  DESTINATION_AIRPORT atraso_medio
  <chr>                <dbl>
1 ABE                  5.80
2 ABI                  4.08
3 ABQ                  5.61
4 ABR                 -3.39
5 ABY                  8.68
6 ACK                  4.91
7 ACT                 10.6
8 ACV                  9.68
9 ACY                 11.8
10 ADK                 -3.19
# i 312 more rows

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela flights;

A chave em flights é DESTINATION_AIRPORT.

b. Identifique a coluna que é a chave na tabela airports;

A chave em airports é IATA_CODE.

c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?

```

flights %>%
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

```

```
# A tibble: 0 x 2
# i 2 variables: DESTINATION_AIRPORT <chr>, atraso_medio <dbl>
```

Não há nenhum aeroporto listado em `flights` que está ausente em `airports`.

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;

e. Armazene a tabela resultante no objeto `flights`.

```
flights <- flights %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

flights
```

```
# A tibble: 322 x 6
  DESTINATION_AIRPORT atraso_medio CITY      STATE LATITUDE LONGITUDE
  <chr>                <dbl> <chr>    <chr>    <dbl>    <dbl>
1 ABE                  5.80 Allentown PA        40.7     -75.4
2 ABI                  4.08 Abilene   TX        32.4     -99.7
3 ABQ                  5.61 Albuquerque NM        35.0    -107.
4 ABR                 -3.39 Aberdeen SD        45.4     -98.4
5 ABY                  8.68 Albany   GA        31.5     -84.2
6 ACK                  4.91 Nantucket MA        41.3     -70.1
7 ACT                 10.6 Waco      TX        31.6     -97.2
8 ACV                  9.68 Arcata/Eureka CA        41.0    -124.
9 ACY                 11.8 Atlantic City NJ        39.5     -74.6
10 ADK                 -3.19 Adak    AK        51.9    -177.
# i 312 more rows
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports %>%
  count(STATE, name = "num_aeroportos") %>%
  arrange(desc(num_aeroportos))
```

```
# A tibble: 54 x 2
  STATE num_aeroportos
  <chr>         <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
7 CO             10
8 MN              8
9 MT              8
10 NC             8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

a. Carregue o pacote leaflet;

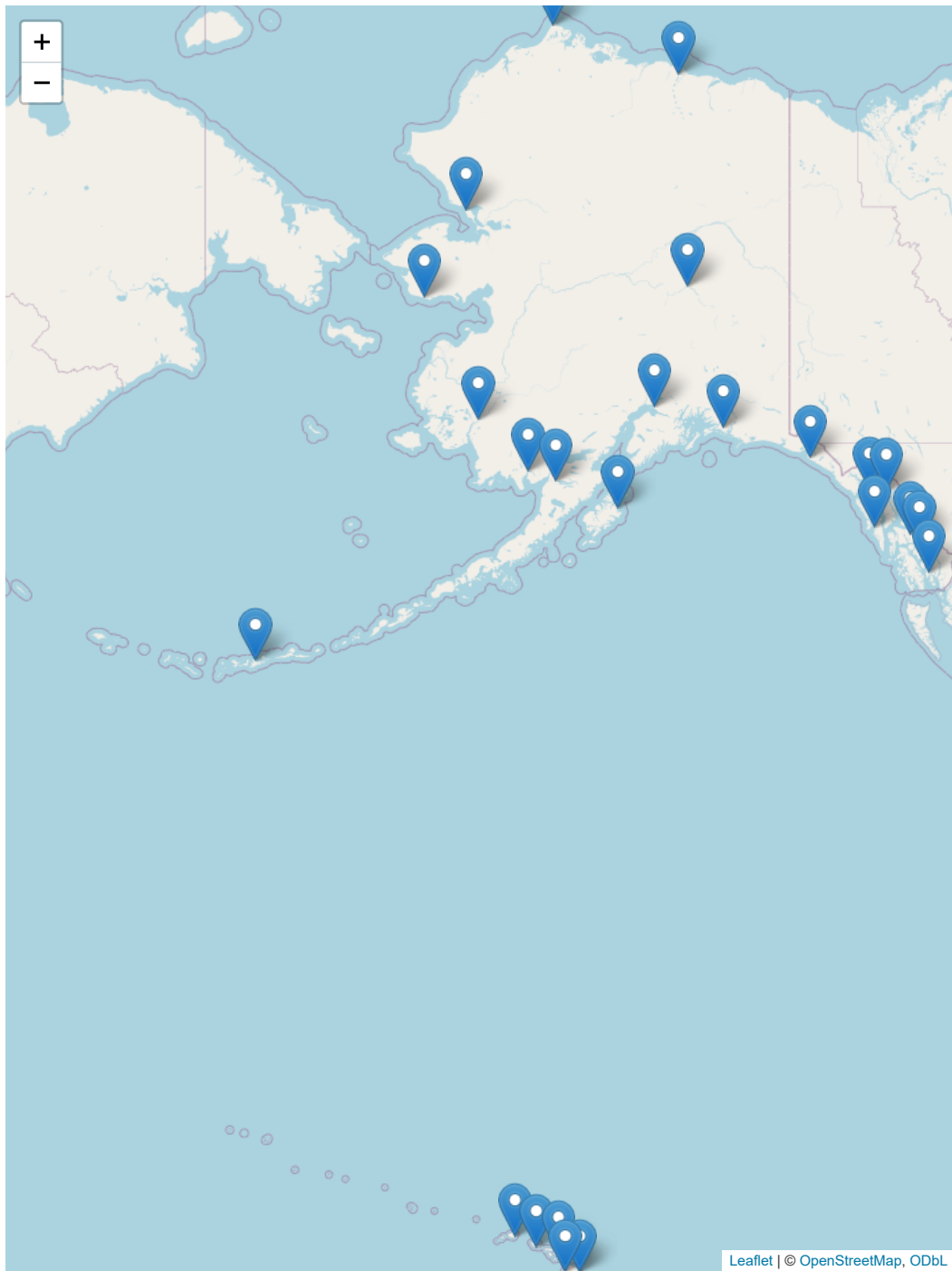
```
library(leaflet)
```

b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;

c. Armazene o grafico em b) numa variável chamada this;

```
this <- flights %>%
  filter(!is.na(LATITUDE), !is.na(LONGITUDE)) %>%
  leaflet() %>%
  addTiles() %>%
  addMarkers(lng = ~LONGITUDE,
             lat = ~LATITUDE,
             popup = ~paste0(CITY, ", ", STATE, "<br>Atraso medio: ",
                             round(atraso_medio, 1), " min"))

this
```



d. Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

Como não estou usando Jupyter, não foi possível realizar esse tópico.

```
sys.setenv(TZ = "America/Sao_Paulo")  
sys.time()
```

```
[1] "2026-09-01 01:00:47 -03"
```